

Seminar 1:

Carlos Buitrago

Table of contents

1	Graphs and Isomorphism	2
1.1	Graphs	2
1.2	Degree of a Vertex	3
1.3	Handshake Lemma	3
1.3.1	Theorem (Handshake Lemma)	3
1.3.2	Proof	3
1.3.3	Corollary	3
1.4	Degree Sequences	4
1.5	7. Graph Isomorphism	4
1.5.1	Definition	4
1.6	Invariants of Isomorphism	5
1.7	Connected Graphs	5
1.7.1	Definition	5
1.8	Trees	5
1.9	Complement of a Graph	6
1.10	Self-Complementary Graphs	6
1.10.1	Proposition	6
1.11	Connectivity of Complements	6
1.11.1	Theorem	6
1.11.2	Proof	7
1.11.3	Corollary	7
1.12	Non-Isomorphic Graphs	7
1.13	Ramsey Theory	7
1.13.1	Definition	7
1.14	How to Verify Whether Two Graphs Are Isomorphic	7
1.14.1	Step 1. Compare the Degree Sequences	8
1.14.2	Step 4. Compare Local Structures	8
1.14.3	Step 5. Match Distinguished Vertices	8
1.14.4	Step 6. Construct an Explicit Bijection	9

1.15 Beyond Graph Invariants: The Graph Isomorphism Problem	9
1.16 The Graph Isomorphism Problem	10
1.16.1 Input	10
1.16.2 Question	10
1.17 A Naive Algorithm	10
1.17.1 How Long Would Brute Force Take?	11
1.18 Complexity-Theoretic Status	12
1.19 The Weisfeiler–Leman Method	12
1.20 Babai’s Breakthrough (2015)	13
1.21 Special Classes of Graphs	13
1.22 Modern Algorithms	14
1.23 State of the Art	14

1 Graphs and Isomorphism

In this seminar we introduce the basic notions of graph theory required for studying graph isomorphism, connectedness, graph complements, and elementary Ramsey theory.

1.1 Graphs

A **simple graph** is a pair

$$G = (V, E),$$

where

- V is a finite set of vertices;
- $E \subseteq \binom{V}{2}$ is a set of unordered pairs of distinct vertices called **edges**.

Throughout the seminar we consider only finite simple graphs.

If $|V| = p$, $|E| = q$, then G is called a (p, q) -**graph**.

The complete graph on n vertices is denoted by K_n .

1.2 Degree of a Vertex

The **degree** of a vertex v is the number of edges incident with v . It is denoted by $d(v)$.

The list

$$(d(v_1), \dots, d(v_n))$$

is called the **degree sequence** of the graph.

Usually the degree sequence is written in non-increasing order.

1.3 Handshake Lemma

One of the most fundamental identities in graph theory is the following.

1.3.1 Theorem (Handshake Lemma)

For every graph G ,

$$\sum_{v \in V(G)} d(v) = 2|E(G)|.$$

1.3.2 Proof

Every edge contributes exactly one to the degree of each of its endpoints. Hence every edge contributes two to the total sum of degrees. Therefore

$$\sum_{v \in V(G)} d(v) = 2|E(G)|. \quad \square$$

1.3.3 Corollary

The number of vertices of odd degree is even.

1.4 Degree Sequences

A sequence

$$(d_1, \dots, d_n)$$

is called **graphical** if there exists a graph whose degree sequence is equal to this sequence.

The Handshake Lemma immediately implies a necessary condition:

$$d_1 + \dots + d_n$$

must be an even number. This condition is necessary but not sufficient. For example, $(4, 4, 4, 4, 2)$ is not graphical.

1.5 7. Graph Isomorphism

Two graphs may look different when drawn in the plane while representing exactly the same combinatorial structure.

1.5.1 Definition

Graphs

$$G = (V, E), \quad H = (W, F)$$

are called **isomorphic** if there exists a bijection $\varphi : V \rightarrow W$ such that

$$uv \in E \iff \varphi(u)\varphi(v) \in F.$$

The map φ is called an **isomorphism**.

We write

$$G \cong H.$$

Intuitively, two graphs are isomorphic if one can be obtained from the other merely by relabelling the vertices.

1.6 Invariants of Isomorphism

A quantity that is preserved under isomorphism is called a **graph invariant**.

If

$$G \cong H,$$

then the following quantities coincide:

- number of vertices;
- number of edges;
- degree sequence;
- number of connected components;
- number of cycles of each length.

Graph invariants are useful for proving that two graphs are not isomorphic.

1.7 Connected Graphs

A **path** in a graph is a sequence of vertices

$$v_0, v_1, \dots, v_k$$

such that every consecutive pair of vertices is connected by an edge.

1.7.1 Definition

A graph is called **connected** if every pair of vertices can be joined by a path.

A maximal connected subgraph is called a **connected component**.

Every graph can be uniquely decomposed into connected components.

1.8 Trees

A graph is called a **tree** if it is connected and contains no cycles.

Trees play a central role in graph theory.

1.9 Complement of a Graph

Let $G = (V, E)$ be a graph. The **complement** of G is the graph $\overline{G} = (V, \overline{E})$, where

$$uv \in \overline{E} \iff uv \notin E.$$

Thus

$$E(\overline{G}) = E(K_n) \setminus E(G).$$

1.10 Self-Complementary Graphs

A graph is called **self-complementary** if

$$G \cong \overline{G}.$$

1.10.1 Proposition

If G is self-complementary and has n vertices, then

$$|E(G)| = \frac{1}{2} \binom{n}{2}.$$

1.11 Connectivity of Complements

1.11.1 Theorem

For every graph G , at least one of the graphs

$$G \quad \text{and} \quad \overline{G}$$

is connected.

1.11.2 Proof

Assume that G is disconnected. Then its vertices can be partitioned into at least two connected components.

Vertices belonging to different components are not adjacent in G , and therefore become adjacent in $\overline{G}$. Consequently every component of G is connected to every other component inside $\overline{G}$, implying that $\overline{G}$ is connected. $\square$

1.11.3 Corollary

Every self-complementary graph is connected.

1.12 Non-Isomorphic Graphs

Two graphs are called **non-isomorphic** if no isomorphism between them exists.

When counting graphs up to isomorphism, graphs differing only by vertex labels are considered identical.

This leads to the notion of an **unlabelled graph**, which represents an entire isomorphism class of labelled graphs.

1.13 Ramsey Theory

Ramsey theory studies the appearance of order in sufficiently large structures.

1.13.1 Definition

The **Ramsey number** $R(s, t)$ is the smallest integer n such that every red-blue colouring of the edges of K_n contains either

- a red copy of K_s , or
- a blue copy of K_t .

1.14 How to Verify Whether Two Graphs Are Isomorphic

Determining whether two graphs are isomorphic is one of the fundamental problems of graph theory.

The trivial step is to compare the number of vertices and edges.

1.14.1 Step 1. Compare the Degree Sequences

Compute the degrees of all vertices and sort them in non-increasing order.

If the resulting sequences differ, the graphs are not isomorphic.

1.14.2 Step 4. Compare Local Structures

Even if two graphs have the same degree sequence, they may still be non-isomorphic.

Useful quantities include:

- number of connected components;
- number of triangles;
- number of cycles of a given length;
- number of vertices of each degree;
- adjacency relations between high-degree vertices.

If any of these quantities differ, the graphs are not isomorphic.

1.14.3 Step 5. Match Distinguished Vertices

Suppose the graphs have passed all previous tests.

Vertices with unique properties must correspond to each other under any isomorphism.

For example, if a graph contains exactly one vertex of degree 4, then this vertex must be mapped to the unique degree-4 vertex of the other graph.

This often greatly reduces the number of possible bijections.

1.14.4 Step 6. Construct an Explicit Bijection

After identifying corresponding vertices, attempt to define a bijection

$$\varphi : V(G) \rightarrow V(H).$$

One must verify that

$$uv \in E(G) \iff \varphi(u)\varphi(v) \in E(H)$$

for every pair of vertices.

If such a bijection exists, then

$$G \cong H.$$

1.15 Beyond Graph Invariants: The Graph Isomorphism Problem

The method based on graph invariants is extremely useful in practice. However, there exist many pairs of non-isomorphic graphs that share all common invariants such as:

- number of vertices;
- number of edges;
- degree sequence;
- number of connected components;
- number of cycles of various lengths.

Consequently, a more systematic approach is required.

This leads to one of the most famous algorithmic problems in graph theory.

1.16 The Graph Isomorphism Problem

1.16.1 Input

Two graphs

$$G = (V, E), \quad H = (W, F)$$

with

$$|V| = |W| = n.$$

1.16.2 Question

Does there exist a bijection $\varphi : V \rightarrow W$ such that

$$uv \in E(G) \iff \varphi(u)\varphi(v) \in E(H)?$$

If such a bijection exists, the graphs are isomorphic.

1.17 A Naive Algorithm

A brute-force approach consists of checking all possible bijections

$$\varphi : V \rightarrow W.$$

Since there are $n!$ such bijections, the running time is approximately $O(n!)$.

Using Stirling's approximation,

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n,$$

which grows extremely rapidly.

1.17.1 How Long Would Brute Force Take?

Suppose, very optimistically, that a modern computer can check 10^9 (one billion) bijections per second. Then the running time is approximately

$$\frac{n!}{10^9}$$

seconds.

For example, $20! \approx 2.4 \cdot 10^{18}$.

Hence brute force would take approximately $\frac{20!}{10^9} \approx 2.4 \cdot 10^9$ seconds.

Since one year has about $3.15 \cdot 10^7$ seconds, this is approximately

$$\frac{2.4 \cdot 10^9}{3.15 \cdot 10^7} \approx 77$$

years.

Thus, even for graphs with only 20 vertices, the naive algorithm is already infeasible.

The situation becomes much worse as n grows.

Number of vertices n	Number of bijections $n!$	Time at 10^9 checks/sec
10	$3.6 \cdot 10^6$	less than 1 second
15	$1.3 \cdot 10^{12}$	about 22 minutes
20	$2.4 \cdot 10^{18}$	about 77 years
25	$1.6 \cdot 10^{25}$	about $5 \cdot 10^8$ years
30	$2.7 \cdot 10^{32}$	about $8.4 \cdot 10^{15}$ years

Therefore, the brute-force method is not a serious algorithm for graph isomorphism, except for very small graphs.

This explains why more sophisticated methods are needed: instead of checking all possible vertex permutations, modern algorithms try to exploit graph structure, vertex refinement, symmetry, and group-theoretic ideas.

1.18 Complexity-Theoretic Status

The Graph Isomorphism problem occupies a unique position in complexity theory.

It is known that

$$\text{GI} \in \text{NP}.$$

Indeed, if an isomorphism exists, it can be verified in polynomial time.

Surprisingly, despite decades of research:

- no polynomial-time algorithm is known for general graphs;
- no proof exists that the problem is NP-complete.

Therefore Graph Isomorphism is one of the few classical problems whose precise complexity remains unknown.

1.19 The Weisfeiler–Leman Method

The most important practical idea is the **Weisfeiler–Leman refinement algorithm**.

The basic version proceeds as follows.

1. Assign every vertex the same color.
2. Repeatedly recolor each vertex according to the multiset of colors of its neighbors.
3. Continue until the coloring stabilizes.

If two graphs produce different color distributions, they cannot be isomorphic.

The algorithm runs in polynomial time.

In practice, it distinguishes most graphs encountered in applications.

1.20 Babai's Breakthrough (2015)

Babai proved that Graph Isomorphism can be solved in

$$2^{(\log n)^{O(1)}}$$

time.

Such running times are called **quasi-polynomial**.

They are significantly smaller than exponential running times such as 2^n but still larger than every polynomial n^k .

1.21 Special Classes of Graphs

Although the complexity of the Graph Isomorphism problem is unknown for arbitrary graphs, polynomial-time algorithms are known for many important graph families.

Examples include:

- trees;
- planar graphs;
- interval graphs;
- bounded-degree graphs;
- graphs of bounded treewidth.

For trees, graph isomorphism can be solved in

$$O(n)$$

time.

Consequently, the difficulty of the Graph Isomorphism problem comes not from sparse or highly structured graphs, but rather from graphs possessing many symmetries.

In practice, special graph classes arise frequently in computer science. Therefore, many real-world instances can be solved much faster than the worst-case theoretical bounds suggest.

1.22 Modern Algorithms

Modern graph isomorphism algorithms do not examine all possible vertex permutations.

Instead, they combine several ideas:

1. Vertex refinement techniques.
2. Partition refinement.
3. Canonical labeling.
4. Backtracking and pruning.
5. Group-theoretic methods.

The most influential practical approach is the Weisfeiler–Leman refinement procedure, which repeatedly partitions vertices into classes according to their local neighborhoods.

These methods eliminate the overwhelming majority of impossible vertex correspondences before any exhaustive search is attempted.

1.23 State of the Art

The current understanding of the Graph Isomorphism problem may be summarized as follows.

1. Graph Isomorphism belongs to the complexity class

NP.

2. No polynomial-time algorithm is known for arbitrary graphs.
3. No proof exists that Graph Isomorphism is NP-complete.
4. Babai’s quasi-polynomial algorithm is the best known theoretical result for general graphs.
5. Polynomial-time algorithms are known for many important graph classes.
6. Practical software can routinely solve instances involving thousands or even millions of vertices.

Consequently, Graph Isomorphism occupies a unique position among classical computational problems: it is neither known to be easy nor known to be hard.